抢红包业务流程

从老板发包到员工抢到的业务设计 (gomall 业务侧 deck)

gomall · 业务侧 deck

今日大纲

抢红包是个什么业务

拆包算法的业务取舍

抢红包链路：复用 gomall 现有套路

过期回收 + 客服话术 + 业务承诺

抢红包是个什么业务

三类典型场景：并发量级差三个数量级

- **一类需求三种规模**：抢的人多 / 包大 / 时间窗短。
- 群红包追求” 分得开心”，平台红包追求” 不出资损”。
- 老板视角：发出 200 元账上必须扣 200，不能多扣也不能漏发。

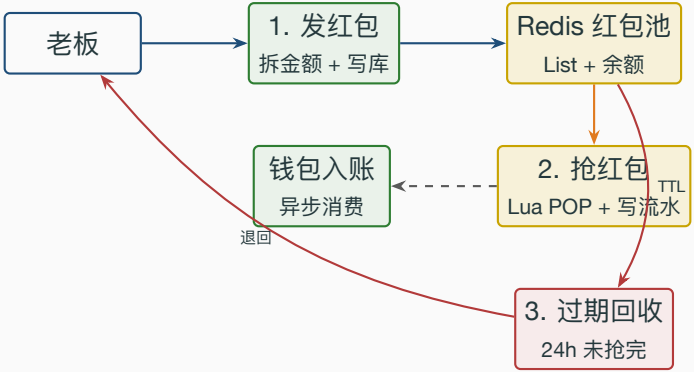
业务规模参考微信红包公开数据（*2024* 年除夕峰值 *6.4* 亿次/分钟）

业务侧的四条硬约束

- **不超发**：账上扣 200 元，红包里加起来不能超过 200 元（哪怕 1 分钱也不行）。
- **不漏发**：用户点了”抢”且系统说”抢到 X 元”，X 必须落到钱包。
- **公平分配**：每人 0.01 元起步，金额分布要有惊喜但不能”一人独大”。
- **防黑产**：单 IP / 单设备 / 同一关系链外的人不能批量薅。

抢库存是”不超卖”，抢优惠券是”不超发”，抢红包是**”不超发 + 不漏发”**——
比库存多一条：抢到的钱必须真正入账，不能停在”已抢”状态。*对照 deck 11 库存承诺、deck 10 流量治理*

发包 / 抢包 / 过期回收：三段独立的状态机



三段流程都**走自己的幂等**：发包用 Idempotency-Key，抢包用 (rpId, userId) 唯一索引，回收用 status 流转。对应 *service/redpacket.go*（*feat/redpacket PR* 即将新增）；当前 *gomall main* 复用 *inventory* + *coupon* 同款 *Lua* + *outbox*

发红包 5 个动作：每一步都必须能补做

1. 校验余额：老板钱包 ≥ 红包总金额。
2. 拆金额：把 200 元拆成 N 份，写入 `redpacket_item` 表。
3. 写主表：`redpacket(id, owner, total, num, status=Open)`。
4. 推 Redis：把每份金额 `RPUSH` 进 `rp:pool:{id}` 列表，TTL 24h。
5. 写 outbox：`redpacket.created` 事件 + 24h 延迟关闭任务。

1–3 同一个 DB tx；4 在 tx 外（Redis 不能参与 2PC）；5 outbox 与主表同 tx。
风险点：第 4 步失败 = 抢的人看到”红包不存在”。补救：消费者读到 `redpacket.created` 时**重做一次推 Redis**。对照 *service/order.go reserve □ tx □ publish* 三段同结构；*service/outbox/publisher.go:64–86* 同款 *outbox* 投递

拆包算法的业务取舍

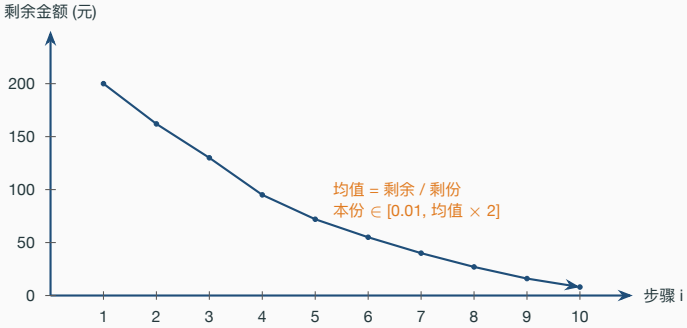
三种拆法的业务后果

算法	表现	业务取舍
平均分	每份 = 总额 / N，无悬念	群红包没人想抢，活动效果差
完全随机	0.01 至总额任意，可能某人独占	不公平，前几名抱怨”剩 0.01 元”
二倍均值法	每份 [0.01，当前均值 × 2]，有惊喜	公平 + 总和精确 + 末位不亏

- 微信 / 支付宝 / 拼多多用的都是**二倍均值法**，工业界事实标准。
- 业务上要的不是”绝对公平”，是**”概率上的惊喜 + 兜底的下限”**——每份 ≥ 0.01 元。
- 服务端拆，不在客户端拆：客户端拆 = 可篡改 = 黑产薅羊毛。

算法源自 *2014* 年微信红包工程实践，公开技术分享多次描述

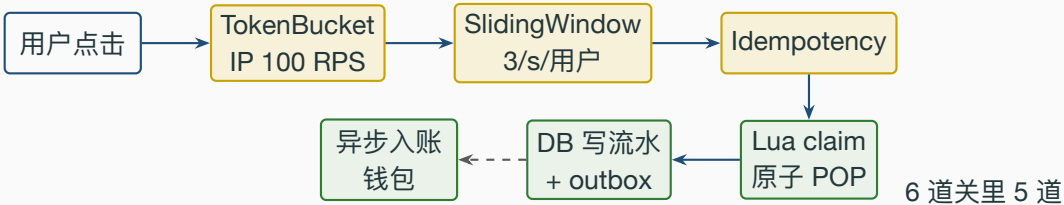
二倍均值法逐次拆分（200 元 / 10 份）



每抢一份后剩余金额单调下降。**2 倍均值上界**保证后面的人还有钱，**0.01 元下界**保证最后一份不空手。拆分算法对应 *service/redpacket.go SplitAmount*（*feat/redpacket PR* 新增）

抢红包链路：复用 gomall 现有套路

一次”抢”经过的 6 道关



在 gomall main 已落地，第 5 道（Lua claim）由 feat/redpacket PR 新增——**复用 inventory.go 同款 Lua 套路**。*routes/router.go:22 / :112–119；middleware/idempotency.go:35–112；repository/cache/inventory.go:32–44*

每道关对应 gomall 的哪段代码

关卡	业务作用	gomall 现有位置
TokenBucket	挡黑产 IP 爬虫	routes/router.go:22
SlidingWindow	挡单用户脚本	middleware/ratelimit.go:59–106
SW Lua	分布式精确计数	repository/cache/ratelimit.go:18–34
Idempotency	网络重试不重领	middleware/idempotency.go:35–112
Lua claim	原子 POP + 防重领	repository/cache/inventory.go:32–44 同款
outbox 入账	钱包异步落账	service/outbox/publisher.go:64–86

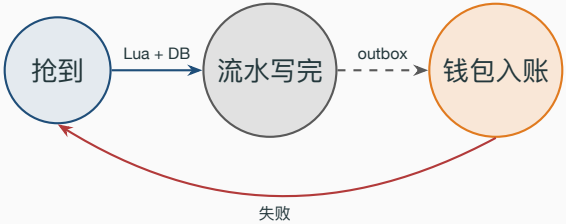
抢红包业务 **80%** 的技术栈在 gomall main 里已经验证过——红包 PR 主要新增的是**业务编排与拆包算法**。除 *Lua claim* 外，其他 *5* 处均为 *gomall main* 在跑的代码

Lua claim 的形状（参照 reserveScript）

```lua
local avail = redis.call('GET', KEYS[1])
if avail == false then return -2 end
local need = tonumber(ARGV[1])
if tonumber(avail) < need then return -1 end
redis.call('DECRBY', KEYS[1], need)
redis.call('INCRBY', KEYS[2], need)
return 1
```

红包 claim Lua 形状**同款**：`HEXISTS rp:claimed:{id} userId` 防重领 → `LPOP rp:pool:{id}` 取一份 → `HSET rp:claimed` 记账 → `INCRBYFLOAT rp:taken`。**一个 EVAL = 原子 POP + 防重领 + 累计。** *repository/cache/inventory.go:32–44 是 claim Lua 模板*

Saga 兜底：抢到了但钱包入账失败



- 抢到 = 写流水 + outbox 事件，**钱包入账走异步**。
- 消费者失败 → MQ requeue / DLX，运营手动处理。
- 极端”始终入账不了”：流水写 Failed，对账退回老板。
- 与 deck 04/11/15 同款：**业务两段，回滚两段**。

service/order_cancel.go:19–63 Saga 模板；service/outbox/publisher.go:64–86

四层防线对照表

与 deck 10 黑产 L1–L4 同结构。gomall 站在 **L1+L2**，覆盖通用 80% 黑产；L3/L4 是红包场景**额外要做**的两件事。*deck 10 § 黑产分层；routes/router.go:22, 112–119*

为什么群红包要做 L4 关系链校验

- 群红包的核心约束：**只有群成员能抢**。
- 不做关系链校验 → 任何人凭 `redpacketId` 就能调 `/redpacket/grab` → 群外人抢光。
- 落点：抢之前先查 `group_member(groupId, userId)`，缺失则拒。
- 与库存 / 优惠券的区别：库存是”先到先得，不限身份”，红包是**”先到先得，但要先验明身份”**。
- gomall 当前没有”群”概念，平台红包不需要 L4；如果接入即时通讯，必须补这道关。

当前 gomall 无 group 模块，L4 是 feat/redpacket PR 引入”群红包”时的新增项

业务码与限流的对应关系

触发场景	gomall 业务码	红包业务码（新增）
TokenBucket	70001	同 70001（IP 限流）
SlidingWindow	70001	同 70001（用户限流）
Idempotency	60002	同 60002（已抢过）
Lua ” 已领”	——	RP-CLAIMED
Lua ” 抢完”	——	RP-EMPTY
Lua ” 过期”	——	RP-EXPIRED

限流 / 幂等**复用现有码**；红包独有的三个状态（已领 / 抢完 / 过期）新增 RP- 前缀业务码。

客服话术由” 业务码 □ 自然语言” 统一映射，前端不解释技术语义。*pkg/e/code.go* 业务码定义；*feat/redpacket PR* 新增 *RP-CLAIMED / RP-EMPTY / RP-EXPIRED*

过期回收 + 客服话术 + 业务承诺

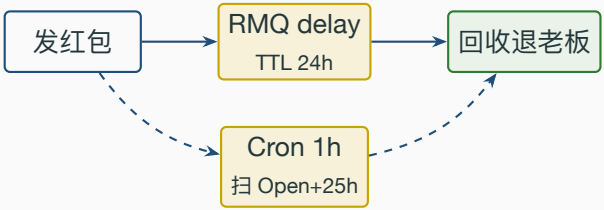
24 小时这条线是怎么定的

方案	用户感受	业务后果
1 小时	多数人来不及看群消息	大量” 未拆”，老板钱被锁
6 小时	上班族下班才能看到	部分场景仍不够
24 小时	跨一个完整作息日	工业界默认（微信 / 支付宝）
72 小时	用户基本能拆	老板资金锁 3 天，资金成本高

24 小时是**用户行为**（睡觉 / 上班 / 不看群）与**资金占用成本**的平衡点。

与 deck 15 的 30 分钟同款逻辑：商家投诉 vs 库存周转，红包是用户体验 vs 资金占用。对照 *deck 15 §1 30* 分钟这条线

双保险：RMQ TTL + Cron 兜底



- RMQ 主路径：发包时**同款**发 24h 延迟消息。
- Cron 兜底：每小时扫 `status=Open AND created_at ≤ now−25h`，比 RMQ 多 1h 容差。
- 与 deck 15 30min/15min 同款双保险——**时长不同，结构一样**。

repository/rabbitmq/order_delay.go:21–72；service/order_task.go:14–48；initialize/cron.go:13–28

回收时把” 未抢部分” 退回给谁

- 已抢的份额（`rp:claimed:{id}`）：**不动**，已经在用户钱包。
- 未抢的份额（`rp:pool:{id}` 残余）：累加 □ 退回**发包人**钱包。
- 退回时机：**过期时**（24h），不是发包时；老板侧的提示文案”X 元未领取已退回”。
- 退回也走 outbox：避免”Redis 桶清零但钱包未到账” 的中间态。

关键不变量：$\sum$ **已抢 + 退回 = 总额**，对账脚本每天凌晨验。
任意时刻这个等式不成立 □ **资损告警**，参考 deck 11 库存对账思路。

repository/cache/inventory.go:127–138 GetStockSnapshot 同款对账接口入口

业务码 → 客服话术 → 用户提示

业务码	内部含义	客服话术
RP-CLAIMED	已经抢过	”您已抢到 X 元，无需再抢”
RP-EMPTY	红包池空	”手慢了，下次再来”
RP-EXPIRED	已过期已回收	”红包过期，金额已退回发包人”
RP-RATELIMIT	限流命中（同 70001）	”活动太火，稍后再试”
RP-NOTINGROUP	L4 关系链校验失败	”您不在此群”

客服

话术**不暴露**”Redis / Lua / 桶 / TTL” 等技术语义。

RP-EXPIRED 文案要明确”已退回发包人”，避免用户找客服要钱。对照 *deck 10 §* 客服话术 *70001/70002/60002/50001* 同结构表

五个角色各看什么指标

角色	关心	看哪个
产品	DAU / 参与率	红包参与率 = 抢人数 / 群人数
运营	大额活动安全	单红包金额上限 / 总额上限
客服	客诉是哪一类	RP-CLAIMED vs RP-EMPTY 占比
SRE	雪崩 / 容量	单红包 QPS 上限 + Redis CPU
风控	黑产形状	IP 分布 / 设备指纹 / 关系链异常

同一份代码、5 个口径，每个口径背后有独立 SLO。

红包是少数**风控权重高**的场景——库存可以靠” 宁可误锁” 兜底，红包资损直接 = 钱。对照 *deck 11 § 角色视图*、*deck 10 § 各角色关心的指标*

与 gomall 现有系统的复用关系

能力	红包用	已存在的同款
Lua 原子扣	抢一份 = POP + 防重领	inventory.go:32–44 / coupon.go:30–44
限流 / 滑窗	3/s/用户	router.go:112–119 / ratelimit.go:59–106
幂等中间件	网络重试不重领	middleware/idempotency.go:35–112
outbox	抢到 □ 钱包入账	service/outbox/publisher.go:64–86
Saga 回滚	入账失败回退状态	service/order_cancel.go:19–63
RMQ 延迟队列	24h 自动回收	rabbitmq/order_delay.go:21–72
Cron 兜底	每小时扫 status=Open	service/order_task.go:14–48

抢红包 ≈ 抢库存 + 抢优惠券 + 订单关闭——业务场景新，技术 80% 复用 gomall 已验证组件。七处全部为 *gomall main* 在跑的代码；*feat/redpacket PR* 主要新增编排

业务承诺总表

承诺	实现	复核
不超发	Lua 原子 POP + INCRBYFLOAT	500 并发抢 100 份零超发
不漏发	outbox + 钱包消费者重试	DLX + 对账脚本
公平拆分	二倍均值法服务端拆	单测分布检查
24h 回收	RMQ TTL 24h	同 deck 15 RMQ 模式
兜底回收	Cron 25h 扫表	同 deck 15 双保险模式
防黑产	TokenBucket + SlidingWindow	deck 10 L1+L2 同款
资金对账	$\sum$ 已抢 + 退回 = 总额	凌晨脚本，偏差告警

七条承诺缺一不可。**复用压测数字**：gomall 现有秒杀链路 30 VU 15s 通过 46 / 期望 45（误差 2.2%），证明限流 + 幂等已在线生效。*stressTest/REPORT.md § 5* 限流中间件 *46/45* 数据；*service/redpacket.go* 在 *feat/redpacket PR* 落地后新增

抢红包业务设计的三个原点

- **资金安全是底线**：超发 / 漏发 / 错入账任何一项都是资损 P0，比库存超卖更严重——库存可以补货，钱补不回来。
- **用户体验是产品**：拆包要有惊喜、抢的瞬间要响应、过期了要明确”钱去哪了”——三句话写进话术。
- **技术 80% 复用**：与库存 / 优惠券 / 订单关闭一脉相承，新增的只是**编排顺序**和**业务文案**。

抢红包不是新技术问题，是**新业务场景**——把 gomall 已验证的组件，按红包的业务节奏重新编排。

业务 Q&A（上）

Q1：拆包为什么不平均，让所有人都开心？

A：平均分用户没动力抢——业务上要的是”惊喜感 + 兜底”。二倍均值法期望仍是均值，方差可控，0.01 元下界保证末位不空手。

Q2：抢完之后能不能给”安慰奖”？

A：业务上可以，但要走**另一笔预算**——不能从原红包池出（违反”不超发”承诺）。常见做法：抢完后弹出”红包封面”等非现金奖励。

Q3：用户网络断了重试，会不会重领？

A：不会。两层兜底——前端带 Idempotency-Key 触发 60002 命中缓存返回原结果；后端 Lua 用 `HEXISTS rp:claimed:{id} userId` 防止同一用户领两次。

业务 Q&A（下）

Q4：过期金额退回时机为什么是 24h 不是即时？

A：用户作息——上班 / 睡觉 / 不看群的用户给一个完整作息日。即时退回会让” 刚发就退”，资金占用反而高（钱包流水翻倍）。

Q5：群红包和平台红包业务区别在哪？

A：群红包要做 L4 关系链校验（只有群成员能抢）+ 限并发（一个群最多几百人）；平台红包没有关系链但要做单 IP / 单设备维度风控（百万级并发）。

Q6：SRE 怎么监控才能提前发现问题？

A：四个看板——单红包 QPS（高于 1k 告警）/ Redis CPU（70% 告警）/ DLX 队列堆积（>10 告警）/ 对账脚本偏差（> 1 分钱告警）。

代码位置一览

全局 TokenBucket *routes/router.go:22*

秒杀滑动窗口（红包复用） *routes/router.go:112–119*

滑动窗口中间件 *middleware/ratelimit.go:59–106*

滑动窗口 Lua *repository/cache/ratelimit.go:18–34*

幂等中间件 *middleware/idempotency.go:35–112*

原子 Lua 模板（库存版） *repository/cache/inventory.go:32–44*

原子 Lua 模板（领券版） *repository/cache/coupon.go:30–44*

outbox 投递 *service/outbox/publisher.go:64–86*

outbox DAO *repository/db/dao/outbox.go:26–79*

Saga 取消模板 *service/order_cancel.go:19–63*

RMQ 延迟队列模板 *repository/rabbitmq/order_delay.go:21–72*

Cron 兜底模板 *service/order_task.go:14–48；initialize/cron.go:13–28*

红包业务编排 *service/redpacket.go 在 feat/redpacket PR 落地后新增*